



Sri Lanka Institute of Information Technology

SE3110

LECTURE 1

Introduction to Software Quality & Testing

Faculty of Computing · Department of Software Engineering



What this lecture covers

01 **Software quality**

02 **Software testing**

03 **Faults & failures**

04 **Why we test**

05 **Cost of defects**

06 **Maturity levels**

07 **Verification & validation**

08 **How we test**

09 **Types of testing**

10 **Testing principles**

11 **The STLC**



01

SECTION 01

Software Quality

Two views of quality — and where testing fits in.

What is software quality?

PRODUCER VIEW · Philip Crosby

Quality is conformance to requirements.

Example: A car is specified for 200 km/h and 15 km/l. If it meets the spec, that is quality.

CUSTOMER VIEW · Joseph Juran / W. E. Deming

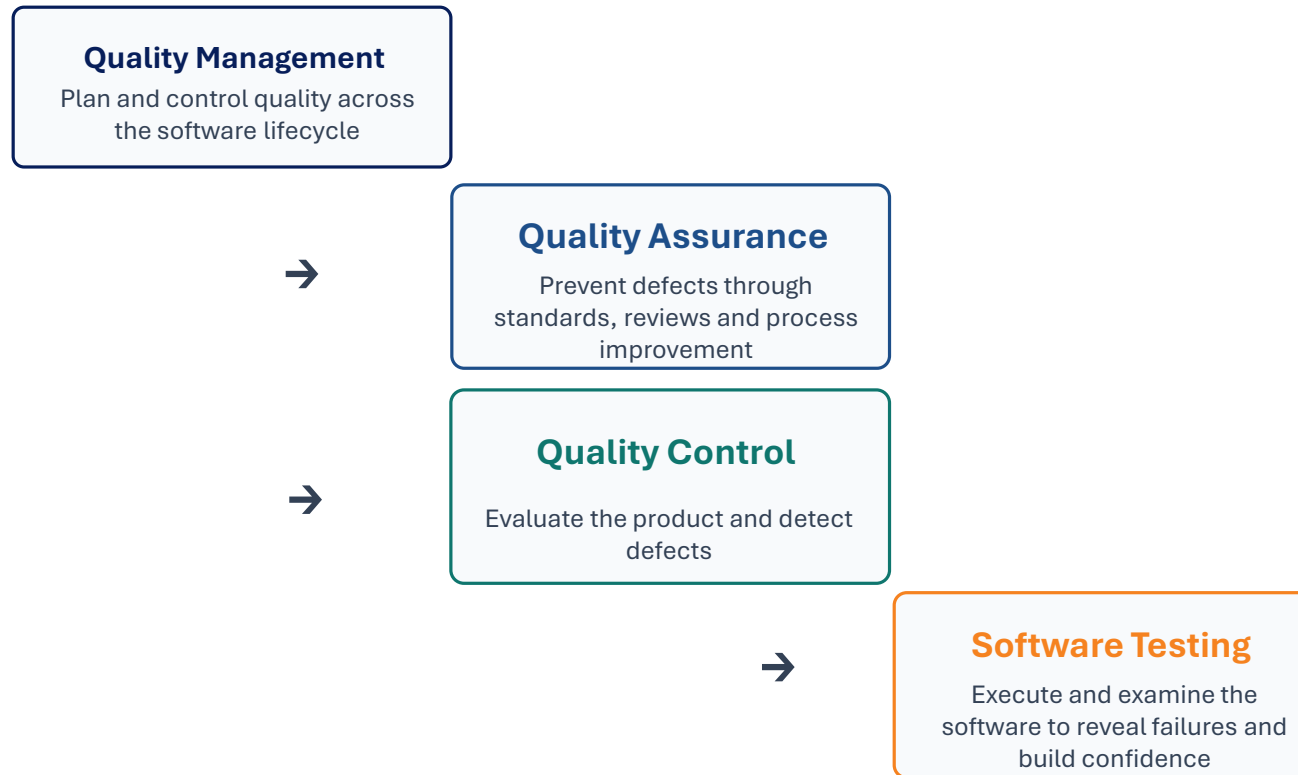
Quality is fitness for use.

Example: A phone with great specs still is not quality if it overheats or freezes for users.

Both matter: good software must conform to its specification and satisfy real users.

How software testing supports software quality

Testing is not the whole quality-management process — it provides evidence for product quality.



What is software testing?

Software testing is the process of evaluating a software system to reveal failures, expose defects, and check whether quality requirements are being met.

Evaluate

Exercise the software against what it should do.

Reveal failures

Observe wrong behaviour before real users experience it.

Assure quality

Give evidence that the product is ready to proceed or ship.



SECTION 02

Faults, Errors & Failures

Precise words for imprecise things — because “bug” means too many things.

The term “Bug”

- *Bug* is used informally
- Sometimes speakers mean fault, sometimes error, sometimes failure ... often the speaker doesn't know what it means !
- This class will try to use words that have precise, defined, and unambiguous meanings

Fault , Error , Failure

Fault

A static defect in the software — a mistake sitting in the source code.

Error

An incorrect internal state, produced when a fault is executed.

Failure

Observable wrong behaviour, relative to the requirements.

Medical analogy

- Symptoms = failures reported by the patient
- Ailment = root cause diagnosed by the doctor
- High BP or arrhythmia = anomalous internal readings

Software faults are present from the start; they do not “wear in” like hardware.

Same fault, different outcome

One passing test proves very little — the fault may not propagate to the output.

```
public static int numZero(int[] arr) {  
    int count = 0;  
    for (int i = 1; i < arr.length; i++) {  
        // FAULT: should start at 0  
        if (arr[i] == 0) {  
            count++;  
        }  
    }  
    return count;  
}
```

Test 1: arr = [2, 7, 0]

Expected 1 · Actual 1 → no failure. The skipped index 0 is not a zero, so the error does not reach the output.

Test 2: arr = [0, 2, 7]

Expected 1 · Actual 0 → failure. The skipped index 0 holds the only zero, so the output is wrong.



SECTION 03

Why Do We Test?

The cost of getting it wrong — from wasted dollars to lost lives.

When testing goes wrong

Mars Climate Orbiter

1999 · \$327M
Metric-vs-imperial unit mix-up sent the probe too low.

Knight Capital

2012 · \$440M
Untested trade-engine deployment lost the sum in 45 minutes.

Ariane 5 Flight 501

1996 · ~\$370M
Conversion overflow forced self-destruct seconds after launch.

THERAC-25

1985–87 · 3 lives
Race condition delivered massive radiation overdoses.

NE Blackout

2003 · 55M people
Alarm-system software error left operators blind.

Boeing 737 MAX

2018–19 · 346 lives
Inadequately tested MCAS contributed to two crashes.

“Software testers try to find the faults before the faults find the users.”

The true cost of poor software

\$22–59B

Estimated annual cost of inadequate software testing to the U.S. economy.
NIST, 2002

\$1.7T

Combined cost of 606 software failures reported worldwide in one year.
Tricentis Fail Watch, 2016

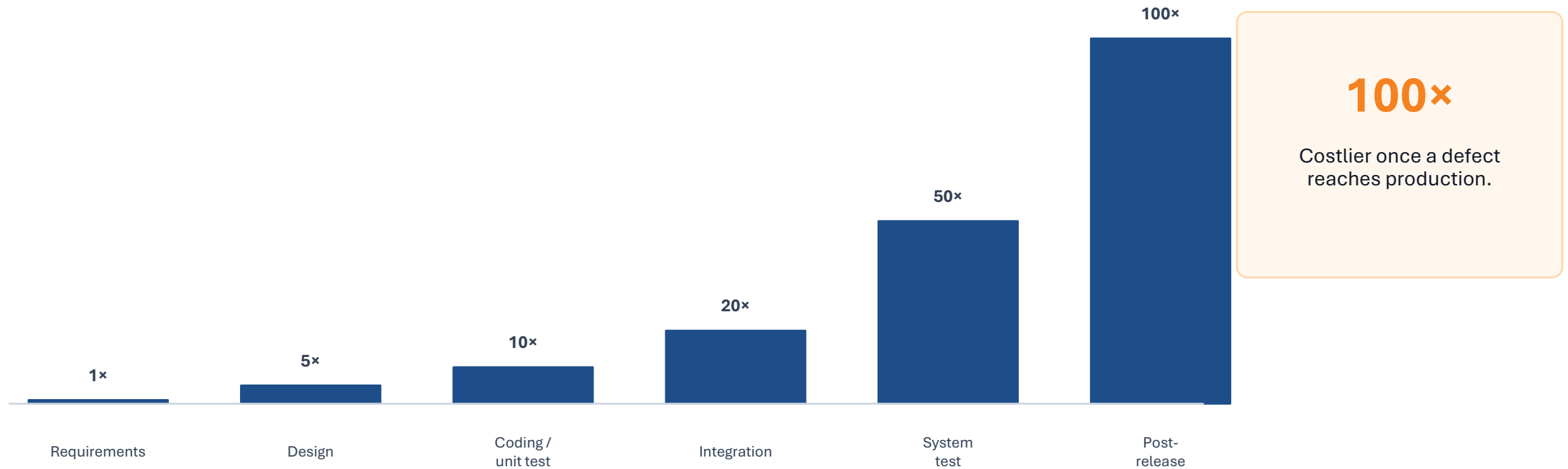
1/2

Of the world's population was affected by those 2016 software failures.
Tricentis Fail Watch, 2016

Poor software is not just frustrating — it is a measurable drag on the global economy.

The later you catch a defect, the more it costs

Find faults early. Test continuously.



Illustrative curve after Boehm and CMU-SEI.

How mature is your testing?

Testing goals evolve as an organisation matures — the aim is Level 4.

- | | | |
|---|--|--|
| 0 | Testing is treated as debugging | No line is drawn between wrong behaviour and mistakes in the code. |
| 1 | Testing tries to show correctness | Passing tests cannot prove complete correctness. |
| 2 | Testing shows failures | A negative stance may pit testers against developers. |
| 3 | Testing reduces risk | Testers and developers cooperate to lower shipping risk. |
| 4 | Testing improves quality | Testing becomes a discipline that lifts the whole team. |

Verification vs Validation

VERIFICATION · Are we building the software right?

Do the outputs of each development phase meet the requirements set in the phase before it?

Focus: correct implementation against the specification.

VALIDATION · Are we building the right software?

Does the finished software actually comply with its intended use?

Focus: real user needs and satisfaction.

Car test: verification checks the design spec; validation checks the customer's actual driving experience.



SECTION 04

How We Test

Approaches, levels, and the qualities we put to the test.

How is testing done?

Two independent questions: who runs the tests, and whether the code is executed.

Manual

A person exercises the app by hand, following a test plan. Useful for UI and exploratory work, but slow and error-prone.

Automated

Scripts and tools run the tests. Fast and repeatable, with an up-front setup cost.

Static

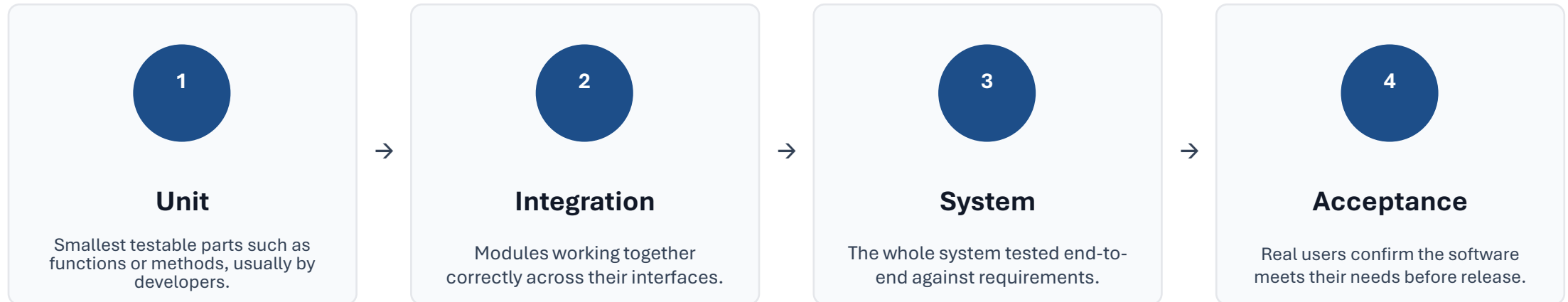
Examine the software without executing it — reviews, inspections and walkthroughs.

Dynamic

Execute the software with real inputs and observe its behaviour.

Test levels

The scope widens from a small unit to the whole product.



Note: Functional and non-functional tests may be performed at several levels.

Functional and non-functional testing

Functional tests ask “does it do what it should?”; non-functional tests ask “how well does it work?”

Functional testing

Checks features and business rules against requirements: login, search, checkout, reports, APIs.

Non-Functional testing

Speed, responsiveness and stability under load — e.g., 10,000 users at once.

Seven principles of testing

1 Testing shows the presence of defects
Never their absence.

3 Test early
Earlier defects are cheaper to fix.

5 Pesticide paradox
Refresh tests to find new defects.

7 Absence of errors is a fallacy
Bug-free software is useless if it is the wrong software.

2 Exhaustive testing is impossible
Prioritise where risk is highest.

4 Defects cluster
A few modules often contain most bugs.

6 Testing is context-dependent
A pacemaker and a game need different testing.



SECTION 05

The Software Testing Life Cycle

A repeatable process — the backbone of Topic 1.

A common Software Testing Life Cycle

Six phases, each guided by entry and exit criteria.

1

Requirement Analysis

Identify what is testable and clarify acceptance criteria.

2

Test Planning

Set scope, strategy, tools, schedule and roles.

3

Test Case Development

Write test cases and scripts; prepare test data.

4

Test Environment Setup

Configure hardware, software and test data.

5

Test Execution

Run tests, log results and report defects.

6

Test Closure

Check exit criteria, report and capture lessons learned.



STLC Quick Challenge

Apply each Software Testing Life Cycle phase to one simple login requirement.

Scenario: A university login system allows students to sign in using a student ID and password. After three incorrect attempts, the account should be temporarily locked.

1 Requirement Analysis

Identify one testable requirement.

2 Test Planning

Decide the feature you will test.

3 Test Case Development

Write one test case.

4 Environment Setup

State what is needed to perform the test.

5 Test Execution

Decide the expected result.

6 Test Closure

State whether the feature is ready for release.

Common challenges in testing

Changing requirements

Time constraints

Complex, large systems

Defect leakage

Need for skilled testers

Key takeaways

Quality has two faces

Conformance for the producer, fitness-for-use for the customer — testing checks both.

Be precise

A fault may cause an error that leads to a failure.

Test early

A defect's cost grows significantly from requirements to production.

Testing ≠ debugging

Aim for Level-4 thinking: testing as a discipline that improves quality.



Over to you

Before you leave, finish this line:

“Today I learned that software testing is ____.”